

Echo Maze

1. Project Overview

Echo Maze is a puzzle-survival maze game developed using Python and Pygame. The player navigates through a grid-based maze, collects checkpoints, and reaches the goal.

The main mechanic of the game is the **Echo System**. During gameplay, the player's movement is recorded. In later rounds, this recorded path is replayed as a Ghost enemy that follows the exact movement of the player from the previous round.

Each stage consists of three rounds:

- **Round 1:** Player explores the maze and movement is recorded.
- **Round 2:** A ghost created from Round 1 appears.
- **Round 3:** Ghosts from Round 1 and Round 2 appear together.

The player must avoid these ghosts while completing the same objective. A stage is cleared after completing Round 3.

2. Project Review

A reference game for this project is **Pac-Man (Namco, 1980)**, which is also a maze-based game where the player collects items while avoiding enemies.

Echo Maze extends this idea with several differences:

- **Enemy behavior:** Pac-Man ghosts chase the player using AI. In Echo Maze, ghosts replay the player's past movements.
- **Game progression:** Pac-Man uses continuous levels, while Echo Maze uses stages with three rounds of increasing difficulty.
- **Strategy:** Echo Maze focuses more on route planning because inefficient paths in earlier rounds create obstacles in later rounds.

3. Programming Development

3.1 Game Concept

Echo Maze is a single-player maze game. The player starts at a spawn point and must complete the tasks (e.g. collecting checkpoints) before the goal becomes accessible.

Gameplay for each stage:

- **Round 1:** No ghosts. Player movement is recorded.
- **Round 2:** A ghost from Round 1 appears.
- **Round 3:** Ghosts from Round 1 and Round 2 appear.

If the player collides with a ghost, the round restarts. The player can also restart the stage to reset the ghost recordings.

The score is calculated when the stage is cleared.

3.2 Object-Oriented Programming Implementation

The game will be implemented using Object-Oriented Programming. The following core classes represent the main components of the game system.

GameManager

This class controls the main game flow, including stages, rounds, and game states.

- Key Attributes: `current_stage`, `current_round`, `game_state`, `player`, `ghosts`, `maze`
- Key Methods: `update()`, `draw()`, `handle_event()`, `start_round()`, `finish_round()`

Player

Represents the player character and handles player movement within the maze. It also records the player's movement path during gameplay.

- Key Attributes: `cx`, `cy` (grid position), `path` (recorded movement)
- Key Methods: `move()`, `update()`, `record_step()`, `get_path()`

Ghost

Represents the ghost enemy created from the player's recorded path. The ghost replays the player's previous movements.

- Key Attributes: `path`, `index`, `cx`, `cy`, `render_x`, `render_y` (pixel position for smooth movement)
- Key Methods: `update()`, `reset()`, `check_collision()`

Map

Stores the maze layout and manages the grid-based environment including walls, paths, and goal positions.

- Key Attributes: `grid`, `tile_size`, `start_position`, `goal_position`
- Key Methods: `draw()`, `is_wall()`, `get_start()`, `get_goal()`

Mission

Handles the objectives within each stage, such as collecting checkpoints and unlocking the goal.

- Key Attributes: `checkpoints`, `collected`, `goal_position`
- Key Methods: `collect_checkpoint()`, `is_complete()`, `update_status()`

3.3 Algorithms Involved

The game uses several basic algorithms:

- **Map Loading**

Maze layouts are pre-designed and stored as 2D grid arrays. Each stage loads a fixed map configuration from a file or hardcoded array. The grid represents walls, paths, player start position, checkpoints, and goal position. The game checks whether a tile is a wall or a path before allowing movement. No procedural maze generation is used. All maze layouts are manually designed and loaded at the start of each stage.

- **Movement Recording**

The player's movement will be recorded by storing the player's grid position (cx, cy) at each step into a list. This list will be used later to create ghost enemies.

- **Replay Algorithm**

The ghost will replay the recorded player path by reading positions from the recorded list step-by-step. The ghost position will update every frame to follow the same path as the player from the previous round.

- **Collision Detection**

Collision detection will be implemented by checking whether the player's grid position is the same as the ghost's grid position. If both positions are equal, a collision is detected and the round restarts.

4. Statistical Data (Prop Stats)

4.1 Data Features

The following gameplay statistics will be recorded for every round (or every event) in each stage, the data variables are:

- **survival_time** – time survived in the round (seconds)
- **steps** – number of steps taken by the player
- **items_collected** – number of checkpoints collected
- **ghost_collisions** – number of collisions with ghosts
- **score** – player score recorded during gameplay
- **round_number** – round number (1, 2, 3)
- **stage_number** – stage number

Feature	Why is it good to have this data? What can it be used for?	How will you obtain 50 values of this feature data?	Which variable (and which class) will you collect this from?	How will you display this feature data (via summarization statistics or via graph)?
survival_time	Used to analyze game difficulty and player survival ability in each round	Recorded automatically at the end of every round. Since each stage has multiple rounds, 100 values can be collected quickly.	survival_time from GameManager	Summary statistics and Bar chart (Average survival time per stage)
steps	Used to measure player movement efficiency and improvement over time	Recorded every round from player movement log. One round generates one value, allowing fast data collection.	steps from Player class	Line graph
items_collected	Used to analyze how well players complete objectives	Recorded every time the player collects an item. One round can generate multiple values, so data can be collected very quickly.	items_collected from Mission class	Summary statistics (mean, min, max)
ghost_collisions	Used to measure game difficulty and player mistakes	Recorded every time a collision with a ghost occurs. This event-based recording allows fast data collection.	ghost_collisions from Ghost class	Boxplot
score	Used to evaluate player performance	Recorded automatically at the end of each round. Each round produces one score value.	score from GameManager	Scatter plot

4.2 Data Recording Method

Gameplay statistics will be stored in a **CSV file (game_data.csv)** using Python's csv module. Each round will generate a new row of data.

4.3 Data Analysis Report

(A) Summary Statistics Table

Feature	Statistical Values to Show in the Table
survival_time	Mean, Median, Min, Max, Standard Deviation
steps	Mean, Median, Min, Max, Standard Deviation
items_collected	Mean, Median, Min, Max, Standard Deviation
ghost_collisions	Mean, Median, Min, Max, Standard Deviation
score	Mean, Median, Min, Max, Standard Deviation

(B) Graphs Specification Table

Feature	Graph Objective	Graph Type	X-axis	Y-axis
survival_time	Compare the average survival time in each stage to analyze game difficulty	Bar Chart	stage_number	Average Survival Time
steps	Analyze player improvement and movement efficiency over rounds	Line Graph	round_number	steps
ghost_collisions	Analyze how ghost difficulty affects player over rounds	Boxplot	round_number	ghost_collisions
score vs steps	Analyze the relationship between movement efficiency and player performance	Scatter Plot	steps	score

The data collected from the game will be analyzed using summary statistics and graphical visualization. Summary statistics such as mean, minimum, maximum, and standard deviation will be used to summarize player performance and game difficulty. In addition, different types of graphs including bar charts, line graphs, boxplots, and scatter plots will be used to visualize the data. The bar chart will be used to compare the average survival time across stages. The line graph will be used to analyze player improvement over rounds. The boxplot will be used to show the distribution of ghost collisions. The scatter plot will be used to analyze the relationship between the number of steps and the score.

5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Implement player movement and maze rendering
3	10	Implement ghost recording and replay system
4	11	Implement mission, stage system and data logging system
5	12	Add UI and polish gameplay
6	13	Testing and debugging
7	14	Submission week (Draft)

6. Document version

Version: 3.0

Date: 27 March 2025

Editor: Panyasiri Aimngern

Date	Name	Feedback
18/03	Peraya	Major Changes Needed - Section 4.1: Your feature Completed (True/False) and Score (one per stage) will take a very long time to reach 100 records if you only record them at the end of a round or

		stage. I am concerned that your current features require too much manual play. - Section 4.3: Your analysis plan is too vague. Saying Results will be presented using tables and charts does not meet the requirement for a useful visualization. You must explain <i>what</i> the chart will show and <i>how</i> it helps analyze the game. - Section 3.3: You mentioned Maze Generation and Replay Algorithm, but provided no technical detail. Briefly specify the algorithm. Minor Changes Needed - Section 3.2: Your classes are well-defined. However, ensure the Ghost class doesn't just store a path, but manages the interpolation logic so the ghost moves smoothly between tiles rather than teleporting from grid to grid.
20/03	Amornrit	Nice extension from normal PAC-MAN and maze games. Each algorithm needs more explanation (e.g., what kind of algorithm will be used to generate a maze). Specify charts you will use.